

BlueBubbles / iMessage Deployment Guide

Revised for Apple Account email identities instead of SIM / phone-number identities

Operating principle: Build and validate one sender identity first. Do not add sender 2, build the bridge, or scale further until the preceding reliability gate passes. Use only recipients who have explicitly opted in; do not manufacture engagement, cycle identities, evade platform enforcement, or reroute around a failing sender.

Phase 1 - Prepare the Mac mini

1. Record the current state

Open Terminal and save the outputs. Record Mac model/RAM, macOS version/build, SIP status, and BlueBubbles version. Keep SIP enabled.

```
sw_vers
uname -m
csrutil status
pmset -g
```

2. Prevent the Mac from sleeping

During the pilot, prevent system/display sleep as appropriate and prevent surprise macOS upgrades. Keep normal security maintenance in mind after testing.

```
sudo pmset -a sleep 0 disksleep 0 powernap 0 tcpkeepalive 1
pmset -g custom
```

Phase 2 - Build the first email-based iMessage identity

3. Prepare Apple Account 01

Create or prepare the first Apple Account and verify its email address. No SIM, cellular number, or dedicated iPhone is required for this email-identity design. The intended email must actually appear as an available iMessage Send & Receive identity after sign-in.

4. Configure Apple Account 01 in Messages

Sign into Messages with Apple Account 01. Open **Messages > Settings > iMessage**. Confirm the intended email appears as an address at which the account can receive messages, then select that email under **Start new conversations from**.

5. Test native iMessage first

Before installing BlueBubbles, send an iMessage from Messages.app to a consenting test recipient and have the recipient reply. Confirm outbound, inbound, blue-bubble delivery, and the correct email sender identity. If native iMessage fails, stop and fix the Apple Account / Messages configuration.

Phase 3 - Create the first macOS user

6. Create imsg01

System Settings > Users & Groups > Add User. Create a Standard user and securely store its password/recovery information.

```
Full Name: iMessage Sender 01
Account name: imsg01
Type: Standard
```

7. Sign into Messages

Log into imsg01, open Messages.app, and sign into Apple Account 01.

8. Configure Messages

Confirm the intended email is reachable through iMessage and choose that email for starting new conversations. Disable Messages in iCloud if following the planned no-sync configuration.

9. Test native Messages on the Mac

Test an existing-chat reply, a completely new conversation, an inbound message, and the sender identity. Do not continue until native Messages works reliably.

```
Existing reply: PASS/FAIL
New conversation: PASS/FAIL
Inbound: PASS/FAIL
Correct email identity: PASS/FAIL
```

Phase 4 - Install BlueBubbles

10. Install BlueBubbles for imsg01

Install the intended BlueBubbles Server build. Grant required permissions, including Full Disk Access, and restart after permission changes.

11. Keep the initial security posture

Start with stock behavior. Private API / SIP-off is a last-resort branch, not the starting configuration.

```
SIP = ON
Private API = OFF
```

12. Configure sender 1

Use a unique server password and the sender-port convention. For the future local bridge, target BlueBubbles over localhost rather than exposing it publicly.

```
sender01 = 12341
sender02 = 12342
sender03 = 12343
sender04 = 12344
sender05 = 12345
```

Phase 5 - Functional acceptance tests

13. Existing conversation test

Send a reply through BlueBubbles into an existing consenting test conversation. Confirm exactly one delivery, the correct email identity, and visibility in Messages.app.

14. Brand-new conversation test

Use the API/server to start a completely new conversation with a consenting test recipient. Confirm chat creation and exactly one delivered message.

15. Inbound event test

Have the recipient reply. Record sender timestamp, Messages.app arrival time, and BlueBubbles detection time.

Phase 6 - Idle reliability tests

16. Run 30/60/120-minute idle tests

Leave Messages and BlueBubbles running without interacting with the Mac. At each interval, send a uniquely labeled inbound test message and measure latency. Require no losses, no duplicates, no manual wake-up, and acceptable latency.

```
IDLE-30
IDLE-60
IDLE-120
```

17. Do not add the keepalive yet

First establish the unmodified baseline. Only add an idle workaround if the baseline demonstrates a real idle failure.

Phase 7 - Failure and recovery testing

18. Wi-Fi interruption

Disconnect Mac Wi-Fi briefly, send a test message, reconnect, and measure automatic recovery. Verify no duplicate event and no required manual Messages restart.

19. Restart BlueBubbles

Quit and relaunch BlueBubbles. Re-test inbound, existing-chat send, and new-chat send.

20. Restart Messages

Restart Messages in a controlled test and verify BlueBubbles resumes correctly.

```
killall Messages
open -a Messages
```

21. Reboot the Mac

After reboot, log into the required macOS user, verify Messages authentication and the email identity, verify BlueBubbles/port health, then run inbound, existing-chat, and new-chat tests. Keep outbound paused until health checks pass.

Phase 8 - Idle workaround, only if needed

22. Test the App Sleep preference

While logged into imsg01, apply the setting, re-login/reboot as appropriate, and repeat the idle tests.

```
defaults write com.apple.iChat NSAppSleepDisabled -bool YES
defaults read com.apple.iChat NSAppSleepDisabled
```

23. Test a per-user keepalive LaunchAgent

If the problem remains, create a conservative per-user LaunchAgent that checks every five minutes that Messages is running without unnecessarily forcing it to the foreground. Keep it only if it measurably improves reliability.

Phase 9 - macOS decision gate

24. PASS branch

Freeze the configuration if existing replies, new chats, inbound events, idle tests, Wi-Fi recovery, app restarts, reboot recovery, outbound echoes, and duplicate checks all pass.

25. New-chat-only failure

If replies work but new-chat creation consistently fails, evaluate the specific upstream fix in an audited source build while keeping SIP enabled. Record exact version, Git commit, applied diff, build date, macOS build, and SIP state.

26. Broad instability

If the selected macOS release is broadly unstable, test another supported macOS configuration in an isolated environment and repeat the same matrix so the OS remains the principal changed variable.

27. Private API last

Only evaluate Private API / SIP-off after the SIP-on paths fail and after weighing security and operational tradeoffs.

Phase 10 - Add sender 2

28. Create imsg02

Repeat the sender-1 process using a separate Apple Account, verified iMessage email identity, Standard macOS user, unique BlueBubbles password, and port 12342. A second SIM or iPhone is not part of this email-based architecture.

29. Keep both user sessions logged in

Use Fast User Switching. Do not log out the background session.

30. Run the 48-hour foreground/background test

Alternate which macOS user is foreground. Record sender, foreground/background state, sent time, Messages arrival, BlueBubbles arrival, and duplicate count. Watch latency, missed events, restarts, RAM pressure, and swap growth. Do not add sender 3 if background reliability is materially worse.

```
vm_stat  
sysctl vm.swapusage  
ps aux | grep -i BlueBubbles
```

Phase 11 - Database and bridge

31. Create the data model

Create senders, conversations, messages, consents, and suppressions. Make provider-originated GUIDs unique and normalize recipient identifiers before suppression/routing checks.

```
create unique index messages_provider_guid_uidx  
on public.messages(provider_guid)  
where provider_guid is not null;
```

32. Make routing sticky

A contact stays pinned to its first sender identity. Never silently reroute to another sender when the assigned sender fails; pause the sender and alert instead.

33. Make sending a state machine

Use queued -> sending -> sent/failed. If a send times out, reconcile against BlueBubbles/provider state before retrying so an accepted message is not duplicated.

34. Preserve per-conversation FIFO

Messages within a conversation must be sent in order, while unrelated conversations may progress independently.

35. Treat inbound events as at-least-once

Deduplicate inbound and Messages-originated outbound echoes by provider GUID. Render the portal from the database; treat BlueBubbles as the provider adapter.

36. Enforce operating/compliance rules in the worker

Before sending, check suppression, valid consent, paused state, sticky-sender match, operating caps, and existing in-flight messages. STOP/DNC must suppress globally according to your policy. Have qualified counsel validate exact consent, identification, and unsubscribe requirements for the jurisdictions you contact.

Phase 12 - Reboot runbook and pilot

37. Reboot runbook

After boot: verify network; log into each required macOS account without logging others out; verify each Messages email identity; verify each BlueBubbles instance; run health checks; then unpause outbound only after required checks pass.

38. Define pilot pass/fail criteria before launch

Track duplicate sends, wrong-sender sends, suppression violations, unreconciled timeouts, inbound latency, new-chat success, background-vs-foreground penalty, manual interventions, reply/opt-out rates, restarts, and cost per qualified conversation versus the control. Scale to senders 3-5 only if predefined gates pass.

Immediate Phase-1 Checklist

- ☐ Apple Account 01 prepared and email verified
- ☐ Email identity appears in Messages Send & Receive
- ☐ Email selected for starting new conversations
- ☐ Native Mac iMessage send/receive works
- ☐ imsg01 Standard user created
- ☐ Messages signed in under imsg01
- ☐ Native Mac new-chat creation works
- ☐ BlueBubbles installed with required permissions
- ☐ SIP remains ON; Private API remains OFF
- ☐ Port 12341 and unique password configured
- ☐ Existing-chat API send passes
- ☐ Brand-new-chat API send passes
- ☐ Inbound event passes
- ☐ 30/60/120-minute idle tests pass
- ☐ Wi-Fi recovery passes
- ☐ BlueBubbles restart passes
- ☐ Messages restart passes
- ☐ Mac reboot/recovery passes
- ☐ No unexpected duplicates

Gate: Do not proceed to sender 2 until the important Phase-1 items are green. This isolates the highest-risk assumption before investing in the portal and bridge.

Architecture: Portal / Supabase worker -> BlueBubbles -> Messages.app -> recipient. Inbound replies travel in reverse. The sender identity is an iMessage-enabled Apple Account email rather than a cellular telephone number.